

76.最小覆盖字符串

1.问题

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard (47.30%)	3232	0	-	-	0

▼ Tags

▼ Companies

给你一个字符串 `s`、一个字符串 `t`。返回 `s` 中涵盖 `t` 所有字符的最小子串。如果 `s` 中不存在涵盖 `t` 所有字符的子串，则返回空字符串 `""`。

注意：

- 对于 `t` 中重复字符，我们寻找的子字符串中该字符数量必须不少于 `t` 中该字符数量。
- 如果 `s` 中存在这样的子串，我们保证它是唯一的答案。

示例 1:

输入: `s = "ADOBECODEBANC", t = "ABC"`
输出: `"BANC"`
解释: 最小覆盖子串 `"BANC"` 包含来自字符串 `t` 的 'A'、'B' 和 'C'。

示例 2:

输入: `s = "a", t = "a"`
输出: `"a"`
解释: 整个字符串 `s` 是最小覆盖子串。

示例 3:

输入: `s = "a", t = "aa"`
输出: `""`
解释: `t` 中两个字符 'a' 均应包含在 `s` 的子串中，因此没有符合条件的子字符串，返回空字符串。

提示：

- `m == s.length`
- `n == t.length`
- `1 <= m, n <= 105`
- `s` 和 `t` 由英文字母组成

进阶：

你能设计一个在

`O(m+n)`

时间内解决此问题的算法吗？

解决方法

好的，我们来分析一下“最小覆盖子串”这个问题，并尝试优化一下分析流程。

1. 问题理解

我们需要在字符串 `s` 中找到一个**最短**的连续子串，这个子串必须包含字符串 `t` 中的**所有**字符，并且每个字符出现的次数**不少于** `t` 中该字符出现的次数。如果找不到这样的子串，就返回空字符串 `""`。

2. 普通人的思路（暴力枚举）

最直接的想法是找出 `s` 的所有可能的连续子串，然后逐个检查它们是否满足条件：

- **生成子串**：用两个循环，一个确定子串的起始位置 `i`，另一个确定结束位置 `j` ($j \geq i$)。这样可以得到所有子串 `s[i...j]`。
- **检查有效性**：对于每个子串 `s[i...j]`，我们需要判断它是否包含了 `t` 中的所有字符，并且数量足够。这通常需要统计子串中每个字符的出现次数，然后和 `t` 中字符的所需次数进行比较。
- **记录最短**：如果一个子串有效，就记录下它的长度。在所有有效的子串中，找到那个长度最短的。

代码思路（伪代码）：

```
minLen = infinity
result = ""

for i from 0 to s.length - 1:
  for j from i to s.length - 1:
    substring = s.substring(i, j + 1)
    if isValid(substring, t): // 检查子串是否包含 t 的所有字符及数量
      if substring.length < minLen:
        minLen = substring.length
        result = substring

return result

function isValid(sub, target):
  // 统计 sub 和 target 中各字符的频率
  // 比较频率，看 sub 是否满足 target 的要求
  // 返回 true 或 false
```

缺点：这种方法非常慢。假设 `s` 的长度是 `m`，`t` 的长度是 `n`。

- 生成所有子串需要 $O(m^2)$ 的时间。
- 检查每个子串的有效性，如果每次都重新统计字符频率，可能需要 $O(m)$ 或 $O(n + |\Sigma|)$ 的时间（其中 $|\Sigma|$ 是字符集大小）。
- 总时间复杂度可能高达 $O(m^3)$ 或 $O(m^2 * n)$ ，对于题目给出的 `m, n <= 10^5` 的数据规模来说，肯定会超时。

3. 更好的方法（滑动窗口）

暴力法的低效在于重复计算。我们可以使用**滑动窗口**技巧来优化。想象一个窗口在字符串 `s` 上滑动：

- **窗口扩张**：我们用两个指针 `left` 和 `right` 定义一个窗口 `[left, right)`（左闭右开）。开始时 `left = right = 0`。不断向右移动 `right` 指针，扩大窗口，将新字符纳入窗口。
- **状态记录**：在窗口扩张过程中，我们需要实时记录窗口内各字符的出现次数，以及这些字符是否满足了 `t` 的要求。通常使用哈希表（或数组，如果字符集有限）来记录：

- `need` : 存储 `t` 中每个字符需要的次数。
- `window` : 存储当前窗口 `[left, right)` 中每个字符的实际次数。
- `valid` : 记录当前窗口中有多少种字符已经满足了 `need` 中的数量要求。
- **窗口收缩** : 当窗口内的字符恰好满足了 `t` 的所有要求 (即 `valid` 等于 `need` 中不同字符的种类数) 时, 我们就找到了一个可行的覆盖子串。这时, 我们尝试收缩窗口, 即向右移动 `left` 指针, 看看能否在保持覆盖 `t` 的前提下, 得到一个更短的子串。
 - 在收缩时, 更新 `window` 计数和 `valid` 状态。
 - 每次收缩前, 记录下当前可行窗口的长度和起始位置, 并与已找到的最小长度比较, 更新最小值。
- **循环** : 重复扩张和收缩的过程, 直到 `right` 指针到达 `s` 的末尾。

算法步骤细化:

1. 初始化 `need` 哈希表, 统计 `t` 中字符频率。
2. 初始化 `window` 哈希表为空, `left = 0`, `right = 0`, `valid = 0` (表示满足 `need` 要求的字符种类数)。
3. 初始化 `minLen = Integer.MAX_VALUE`, `start = 0` (记录最小覆盖子串的起始索引)。
4. **主循环 (扩张窗口)**: 当 `right < s.length()` 时:
 - a. 取 `s[right]` 字符 `c`, 将 `c` 加入 `window`, 更新 `window[c]` 计数。
 - b. `right++`。
 - c. **检查 `c` 是否影响 `valid`**: 如果 `c` 是 `t` 中需要的字符 (`need.containsKey(c)`), 并且 `window[c]` 的值正好等于 `need[c]`, 说明字符 `c` 的数量要求刚刚被满足, `valid++`。
 - d. **内循环 (收缩窗口)**: 当 `valid == need.size()` 时 (表示当前窗口已覆盖 `t`):
 - i. **更新最小子串**: 如果当前窗口长度 `right - left` 小于 `minLen`, 则更新 `minLen = right - left`, `start = left`。
 - ii. 取 `s[left]` 字符 `d`。
 - iii. `left++`。
 - iv. **检查 `d` 是否影响 `valid`**: 如果 `d` 是 `t` 中需要的字符 (`need.containsKey(d)`), 并且 `window[d]` 的值在移出 `d` 之前正好等于 `need[d]`, 说明字符 `d` 的数量要求在移出后将不再被满足, `valid--`。
 - v. 更新 `window[d]` 计数 (减 1)。
5. 循环结束后, 如果 `minLen` 仍然是 `Integer.MAX_VALUE`, 说明没找到覆盖子串, 返回 `""`。否则, 返回 `s.substring(start, start + minLen)`。

时间复杂度: `left` 和 `right` 指针都只遍历 `s` 一次, 哈希表操作近似 $O(1)$ 。初始化 `need` 需要 $O(n)$ 。总时间复杂度为 $O(m + n)$, 满足进阶要求。

4. 代码实现

```
import java.util.HashMap;
import java.util.Map;

class Solution {
    public String minWindow(String s, String t) {
        if (s == null || t == null || s.length() < t.length()) {
            return "";
        }

        // need 存储 t 中字符的需求量
```

```

Map<Character, Integer> need = new HashMap<>();
for (char c : t.toCharArray()) {
    need.put(c, need.getOrDefault(c, 0) + 1);
}

// window 存储当前窗口中字符的数量
Map<Character, Integer> window = new HashMap<>();

int left = 0, right = 0; // 滑动窗口的左右边界 [left, right)
int valid = 0; // 窗口中满足 need 条件的字符种类数

// 记录最小覆盖子串的起始索引和长度
int start = 0;
int minLen = Integer.MAX_VALUE;

while (right < s.length()) {
    // c 是将移入窗口的字符
    char c = s.charAt(right);
    // 右移窗口
    right++;
    // 进行窗口内数据的一系列更新
    if (need.containsKey(c)) {
        window.put(c, window.getOrDefault(c, 0) + 1);
        // 注意：这里用 equals 比较 Integer 对象
        if (window.get(c).equals(need.get(c))) {
            valid++;
        }
    }
}

// 判断左侧窗口是否要收缩
while (valid == need.size()) {
    // 在这里更新最小覆盖子串
    if (right - left < minLen) {
        start = left;
        minLen = right - left;
    }
    // d 是将移出窗口的字符
    char d = s.charAt(left);
    // 左移窗口
    left++;
    // 进行窗口内数据的一系列更新
    if (need.containsKey(d)) {
        // 注意：这里用 equals 比较 Integer 对象
        if (window.get(d).equals(need.get(d))) {
            valid--; // 只有当某个字符刚好满足need时移出，valid才减1
        }
        window.put(d, window.get(d) - 1);
    }
}

// 返回最小覆盖子串，如果没找到则 minLen 保持为初始值
return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);

```

```

    }
}

```

5. 示例演练

`s = "ADOBECODEBANC"` , `t = "ABC"`

- `need = {'A': 1, 'B': 1, 'C': 1}` . `need.size() = 3` .
- `left=0, right=0, valid=0, minLen=inf, start=0` . `window={}` .

right	char <code>c</code>	window	valid	left	char <code>d</code>	Shrink?	Up m
0→1	A	{'A': 1}	1	0	-	No	-
1→2	D	{'A': 1, 'D': 1}	1	0	-	No	-
2→3	O	{'A': 1, 'D': 1, 'O': 1}	1	0	-	No	-
3→4	B	{'A': 1, 'D': 1, 'O': 1, 'B': 1}	2	0	-	No	-
4→5	E	{'A': 1, 'D': 1, 'O': 1, 'B': 1, 'E': 1}	2	0	-	No	-
5→6	C	{'A': 1, 'D': 1, 'O': 1, 'B': 1, 'E': 1, 'C': 1}	3	0	-	Yes	6
				0→1	A	Yes	6
			2	1→2	D	No	-
6→7	O	{'O': 2, 'B': 1, 'E': 1, 'C': 1}	2	2	-	No	-
7→8	D	{'O': 2, 'B': 1, 'E': 1, 'C': 1, 'D': 1}	2	2	-	No	-
8→9	E	{'O': 2, 'B': 1, 'E': 2, 'C': 1, 'D': 1}	2	2	-	No	-
9→10	B	{'O': 2, 'B': 2, 'E': 2, 'C': 1, 'D': 1}	2	2	-	No	-
10→11	A	{'O': 2, 'B': 2, 'E': 2, 'C': 1, 'D': 1, 'A': 1}	3	2	-	Yes	11
				2→3	O	Yes	11
				3→4	B	Yes	11
				4→5	E	Yes	11
				5→6	C	Yes	11
			2	6	-	No	-
11→12	N	{'B': 1, 'E': 1, 'D': 1, 'A': 1, 'N': 1}	2	6	-	No	-
12→13	C	{'B': 1, 'E': 1, 'D': 1, 'A': 1, 'N': 1, 'C': 1}	3	6	-	Yes	13
				6→7	O	Yes	13
				7→8	D	Yes	13

right	char c	window	valid	left	char d	Shrink?	Update min
				8→9	E	Yes	13
				9→10	B	Yes	13 10 9= WI 13 m w v {A
			2	10	-	No	-

right reaches end. minLen = 4, start = 9.

Return

s.substring(9, 9 + 4) which is "BANC".

找到具有 1 个许可证类型的类似代码